Computer Architecture (Practical Class)
Assembly: Arrays, Strings and Loops

Luís Nogueira Nuno Pereira Raquel Faria

Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

{lmn,nap,arf}@isep.ipp.pt

2018/2019

Arrays - Review (1/4)

Arrays

- Are a continuous memory area
- Each element of the array occupies the number of bytes determined by its data type

Array Declaration Examples (in C)

```
char           array_a[12];
char           *array_b[8];
long long int  array_c[6];
long long int  *array_d[5];
int            array_e[] = {10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60};
```

- What is the size of each element ?
- What is the total size ?

Arrays - Review (2/4)

Arrays

- Are a continuous memory area
- Each element of the array occupies the number of bytes determined by its data type

Array Declaration Examples (in C)

```
char            array_a[12];
char            *array_b[8];
long long int   array_c[6];
long long int   *array_d[5];
int             array_e[] = {10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60};
```

- What is the size of each element ?
- What is the total size ?

Array	Element Size	Total Size
array_a	1	12
array_b	4	32
array_c	8	48
array_d	4	20
array_e	4	44

Declaring Arrays in C and Assembly

Array Declaration in C

```c
// uninitialized array
long long int array_c[6];

// initialized array
int array_e[] = {10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60};
```

Array Declaration in Assembly

```
.section .bss          # section BSS (uninitialized variables)

.comm array_c, 48      # space for 6 long long (6x8), name: array_c

.section .data         # section data (initialized variables)

array_e:               # name: array_e
    .int 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60 # array initialization
```

Declaring Strings (Arrays of char) in C and Assembly

String (Array of char) Declaration in C

```
// uninitialized string (array of char)
char str_a[10];

// initialized string (array of char)
char str_b[] = "computer architecture";
```

String (Array of char) Declaration in Assembly

```
.section .bss          # section identifier

.comm str_a, 10        # space for 10 bytes; variable name: str_a

.section .data         # section identifier

str_b:                 # variable name
    .asciz "computer architecture"
```

Arrays - Accessing values (1/5)

Pointer in C

```
// assign x the value pointed by p
// p is a pointer to an array of int
int x = *p;
```

What is the equivalent Assembly code?

- We have seen that an array is a continuous area in memory, where each element occupies the number of bytes determined by its data type
- We can store memory addresses in registers
- When a register stores a memory address, it is equivalent to a pointer in C

Arrays - Accessing values (2/5)

```
movX (register),destination
```

Indirect addressing

- When we use the register as a pointer by placing it between parentheses

Important notes:

- Byte addressing (addresses are manipulated byte by byte)
- To move the pointer to the next element, we must know the size of the element
- *Must* use the correct `mov` variant (`movl`, `movw`, `movb`) according to the number of bytes that you want to copy
- We cannot do `movl (%ebx),(%eax)` — Why?

Arrays - Accessing values (3/5)

Example: Accessing the 10th element of an array of `int`

Return the value of the 10th element (`http://codepad.org/TzmXLvAi`)

```
# copy the value of the 10th element of array_a to EAX
.section .data

array_a:
        .int 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60

.section .text

.global tenth_element      # declare function as global
tenth_element:             # function start
# prologue
    pushl %ebp             # save previous stack frame pointer
    movl %esp, %ebp        # the stack frame pointer for our function

# body
    movl $array_a, %edi    # copy the address of array_a to %edi
    addl $36, %edi         # move to 10th element ((10-1)*4=36)
    movl (%edi),%eax       # copy the value pointed by %edi to %eax
                           # this will be our return value
# epilogue
    movl %ebp, %esp        # restore the stack pointer ("clear" the stack)
    popl %ebp              # restore the stack frame pointer
    ret                    # return from the function
```

Arrays - Accessing values (4/5)

Example: Accessing the 5th element of a string (an array of `char`)

Return the value the 5th char (`http://codepad.org/VJBvSlvn`)

```
# copy the value of the 5th char of str in AL
.section .data

str:
        .asciz "computer architecture"

.section .text

.global fifth_char
fifth_char:
# prologue
    pushl %ebp              # save previous stack frame pointer
    movl %esp, %ebp         # the stack frame pointer for our function

# body
    movl $str, %ebx         # copy address of str to ebx
    addl $4, %ebx           # move to the fifth element
    movb (%ebx),%al         # copy the char value pointed by ebx to al

# epilogue
    movl %ebp, %esp         # restore the stack pointer ("clear" the stack)
    popl %ebp               # restore the stack frame pointer
    ret                     # return from the function
```

Arrays - Accessing values (5/5)

```
movX offset(register),destination
```

Indirect Addressing with Offset

- When we add a constant value to the address stored in a register

Examples:

```
movl 4(%ebx),%eax     # ebx is a pointer to an array of integers
```

- Places the value contained in the memory location 4 bytes after the location pointed to by the EBX register in the EAX register
- It is equivalent to:

```
addl $4, %ebx
movl (%ebx),%eax
subl $4, %ebx     # the value of ebx is not modified
```

```
movl -8(%ebx),%eax     # ebx is a pointer to an array of integers
```

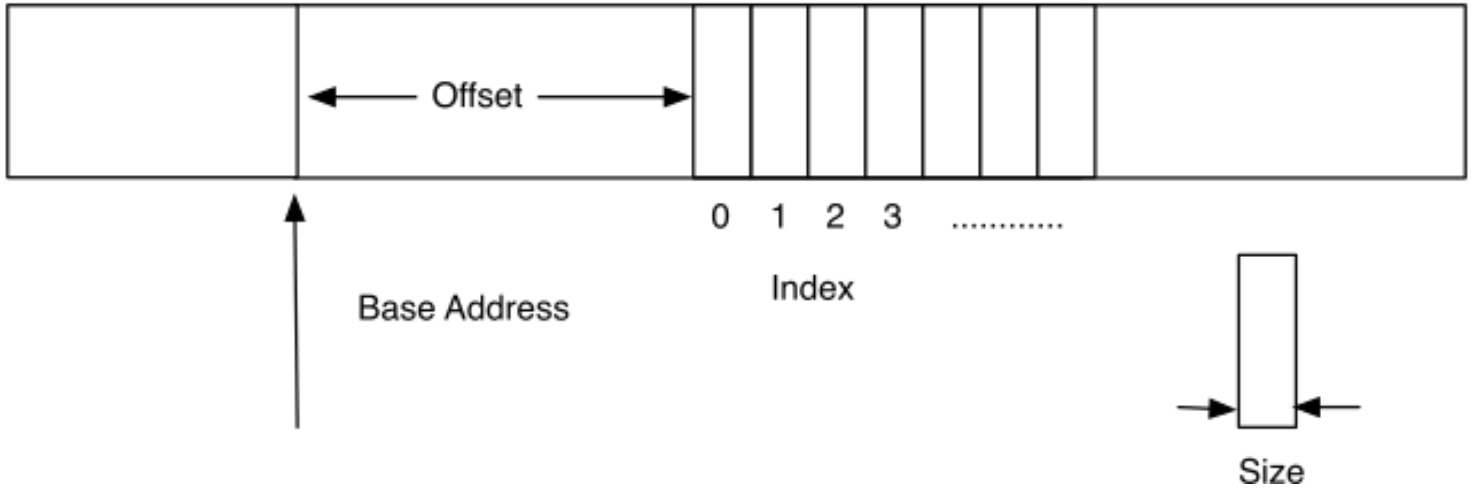
- It is equivalent to:

```
subl $8, %ebx
movl (%ebx),%eax
addl $8, %ebx     # the value of ebx is not modified
```

Indexed memory mode (1/3)

When accessing data in an array, one can use an index system to determine which value you are accessing. For this, we define:

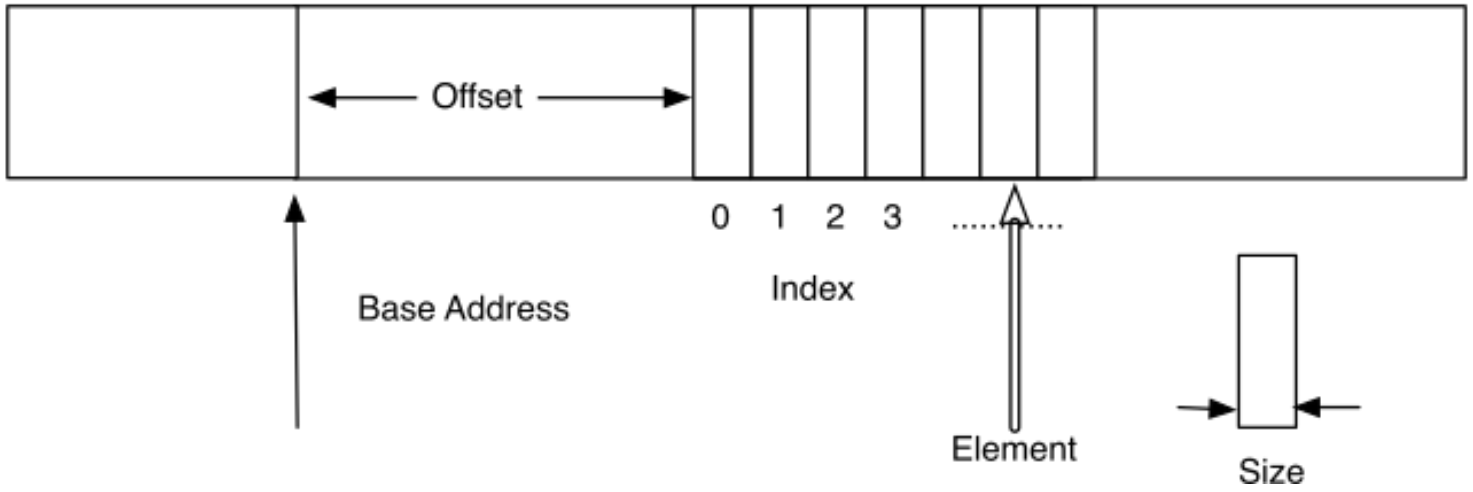
1. `base_address`: a pointer to the start of the memory we want to address
2. `offset`: a displacement value between the beginning of the memory and when we start counting elements
3. `size`: the size of each element
4. `index`: the index of the element we want to access



Indexed memory mode (2/3)

To calculate the address, calculate the expression:

```
base_address+offset+index*size
```



- This is what the *indexed memory mode* does

Indexed memory mode (3/3)

`offset (base_address, index, size)`

- `base_address`: A base address (register)
- `offset`: An offset address to add to the base address (literal)
- `index`: An index to determine which data element to select (register)
- `size`: The size of the data element (literal with value 1, 2, 4, or 8)

Notes:

- If any of the values are zero, they can be omitted (but the commas are still required as placeholders).
- If we omit the `base_address`, the `offset` can be an absolute address

Indexed Memory Mode Example

```
# section identifier
.section .data

# declare integer array named 'array_a'
array_a:
    .int 10, 15, 20, 25, 30, 35

# section identifier
.section .text

# function implementation
function_example:
        ...

        # index to access: 3rd element
        movl $2, %edi

        # move value to %eax
        # note: base_address is omitted
        movl array_a(, %edi, 4), %eax

        ...

        ret
```

Practice

What is the Assembly code that implements the expression given?

Assume:

- an array `int array_e[10]`, with its initial address in `edx`
- the value of *i* (an integer) was previously copied to `ecx`

Expression	Type	Assembly
eax ← array_e	`int *`	
eax ← array_e[0]	`int`	
eax ← array_e[i]	`int`	
eax ← *(array_e+i-3)	`int`	

Practice

What is the Assembly code that implements the expression given?

Assume:

- an array `int array_e[10]`, with its initial address in `edx`
- the value of *i* (an integer) was previously copied to `ecx`

Expression	Type	Assembly
eax ← array_e	`int *`	`movl %edx,%eax`
eax ← array_e[0]	`int`	`movl (%edx),%eax`
eax ← array_e[i]	`int`	`movl array_e(, %ecx, 4), %eax`
eax ← *(array_e+i-3)	`int`	`movl -12(%edx,%ecx,4),%eax`

The for loop in Assembly

For Loop Example

```
# function that implements a for loop: for(i=0; i<limit; i+=n)
# Using conditional jumps
function_example:
  ...

# function body
  movl limit, %eax    # limit value in %eax
  movl $0, %ebx       # iterating index in %ebx (i=0)

  my_loop:            # loop start
    cmpl %eax, %ebx   # compare index with limit value
    jge end_my_loop   # jump if i >= limit

    ...               # loop body

    addl n, %ebx      # increments index (i+=n)
    jmp my_loop

  end_my_loop:

  ...

  ret
```

The Loop, Loope, Loopz, Loopne, Loopnz Instructions

`loop label`

- Automatically decrements %ecx and jumps to the label if %ecx is different from 0

`loop` **Instruction Example**

```
    ...
    movl $100, %ecx
my_loop:
    ...
    loop my_loop
    ...
```

- `loope/loopz`: decrements %ecx and jumps to the label if %ecx is different from 0, *and the flag ZF is active*

- `loopne/loopnz`: decrements %ecx and jumps to the label if %ecx is different from 0, *and the flag ZF is* ***not*** *active*

Important Notes:

- The `loop` instructions test the flags, but do not change them
- What will happen if the %ecx register is zero before the first call to `loop` ?

Example: Iterate through a string

Loop Example: Iterate String

```
.section .data
str:
        .asciz "computer architecture"

.section .text
.global iterate_string # declare function as global
iterate_string:        # function start
# prologue
  ...

# body
  movl  $str, %ebx     # address of string in %ebx (notice the $)
string_loop:
  movb  (%ebx),%cl     # copy char pointed by %ebx to %cl
  cmpb  $0,%cl         # check if char is zero (end of string)
  je    end_loop       # no more chars in string

  ...                  # do something with char...

  incl   %ebx          # moves pointer to next 1 byte
  jmp    string_loop   # jumps to next iteration

end_loop:              # we reached the end of the string...

# epilogue
  ...
```

Example: Count the number of chars in a string

Count Number of Chars Example (http://codepad.org/TrqqYuK4)

```
.section .data          # section identifier (data)
str:                    # declare string
   .asciz "computer architecture"

.section .text          # section identifier (text)
str_count:              # function start
# prologue
    pushl %ebp          # save previous stack frame pointer
    movl %esp, %ebp     # the stack frame pointer for our function
# body
    movl  $str,%esi     # copy str address to %esi (notice the $)
    movl  $0,%eax       # counter = 0
cnt_loop:
    movb (%esi),%cl     # copy char from str1 (pointed by %esi) to %cl
    cmpb $0,%cl         # check if this is the end of the string
    jz cnt_loop_end     # jump if it is the end
    incl %eax           # counter ++
    incl %esi           # move to the next char in str1
    jmp cnt_loop        # next iteration
cnt_loop_end:
    # note: return value (counter) in %eax
# epilogue
    movl %ebp, %esp     # restore the stack pointer ("clear" the stack)
    popl %ebp           # restore the stack frame pointer
    ret                 # return from the function
```

Practice: `str_copy()` in Assembly

- Implement a function `int str_copy()` that copies a string from `ptr1` to `ptr2` (two global pointers to char). Test it by calling your function from a C program.
- The function should return the number of chars copied
- Consider that the variables are declared in assembly and initialized in C.

Practice: str_copy() in Assembly - C Source (1/2)

str_cpy.h (http://codepad.org/fSwYx1fu)

```
#ifndef _STR_CPY_ // avoid duplicate definitions
#define _STR_CPY_

// maximum number of chars in a string
#define MAX_CHAR 20

// function implemented in Assembly
int str_copy();

// pointers declared in Assembly
extern char *ptr1, *ptr2;

#endif
```

Practice: str_copy() in Assembly - C Source (2/2)

main.c (http://codepad.org/kG0cWc48)

```c
#include <stdio.h>    // for printf()
#include "str_copy.h" // definition of MAX_CHAR, ptr1, ptr2, str_copy()

int main (void){
    char str1[MAX_CHAR] = "abcdef";
    char str2[MAX_CHAR];     // important: str2 must have space for str1
    int n_chars;

    // assign address of strings to global pointers, defined in assembly
    ptr1 = str1;
    ptr2 = str2;

    // call function str_copy(), implemented in Assembly
    n_chars = str_copy();

    // output results
    printf("Copied %d chars\n",n_chars);
    printf(" str1 = %s\n", str1);
    printf(" str2 = %s\n", str2);

    return 0;
}
```

Practice: str_copy() in Assembly - Assembly Source (1/2)

str_copy.s (http://codepad.org/Gl0p6Mwd)

```
.section .bss              # section identifier (bss)

.comm   ptr1,4             # declare pointer (4 bytes)
.comm   ptr2,4             # declare pointer (4 bytes)
.global ptr1               # declare ptr1 global so that it can be used in C
.global ptr2               # declare ptr2 global so that it can be used in C

.section .text             # section identifier (text)

.global str_copy           # declare function as global so that it can be used in C
str_copy:                  # function start
# prologue
    pushl %ebp             # save previous stack frame pointer
    movl %esp, %ebp        # the stack frame pointer for our function
# body
    movl  ptr1,%esi        # copy str1 address to %esi
    movl  ptr2,%edi        # copy str2 address to %esi
    movl  $0,%eax          # counter = 0
```

Practice: str_copy() in Assembly - Assembly Source (2/2)

str_copy.s (http://codepad.org/Gl0p6Mwd)

```
str_loop:
    movb (%esi),%cl      # copy char from str1 (pointed by %esi) to %cl
    movb %cl,(%edi)      # copy char in %cl to str2 (pointed by address in %edi)
    cmpb $0,%cl          # check if this is the end of the string
    jz str_loop_end      # jump if it is the end
    incl %eax            # counter ++
    incl %esi            # move to the next char in str1
    incl %edi            # move to the next char in str2
    jmp str_loop         # next iteration

str_loop_end:            # loop end; note: return value (counter) in %eax
# epilogue
    movl %ebp, %esp      # restore the stack pointer ("clear" the stack)
    popl %ebp            # restore the stack frame pointer
    ret                  # return from the function
```